Task-2 :- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and technical requirements for automating the lifecycle of AWS EC2 instances and ECS Fargate services using Jenkins Pipelines. The intended audience includes DevOps engineers, Cloud Architects, and stakeholders interested in cost optimization and infrastructure automation.

1.2 Scope

- **In-Scope:** Provisioning tagged EC2 instances, configuring Jenkins with AWS credentials, creating declarative pipelines for Start/Stop actions, and extending automation to ECS Fargate service scaling.
- **Out-of-Scope:** Auto-scaling group configurations, VPC/Network creation (assumed existing), and application-level deployment logic.

1.3 Definitions, Acronyms, and Abbreviations

- **EC2:** Amazon Elastic Compute Cloud
- **ECS:** Amazon Elastic Container Service
- **Fargate:** Serverless compute engine for containers
- **IAM:** Identity and Access Management
- **CLI:** Command Line Interface

1.4 References

- AWS CLI Command Reference (EC2/ECS)
- Jenkins Pipeline Documentation
- *Bonus Reference:* "Automatically Stop ECS Fargate Services with AWS Lambda" (Darnel Edwards).

1.5 Overview

This document transitions from project objectives to specific functional use cases, followed by security and reliability requirements, concluding with a risk assessment for the automation workflow.

---

2. Project Overview

2.1 Project Objectives

- Implement tag-based management for EC2 to avoid hardcoding IDs.
- Centralize environment control (Dev/Prod) within Jenkins.
- Automate Fargate scaling to zero to minimize serverless costs during idle hours.

2.2 Business Goals

- **Cost Reduction:** Minimize AWS spend by stopping non-production resources after hours.
- **Operational Efficiency:** Provide a "one-click" interface for developers to manage their environments.

2.3 Background

Current manual intervention for starting/stopping resources is prone to error and inconsistency. Leveraging Jenkins allows for scheduled tasks and audit trails for infrastructure changes.

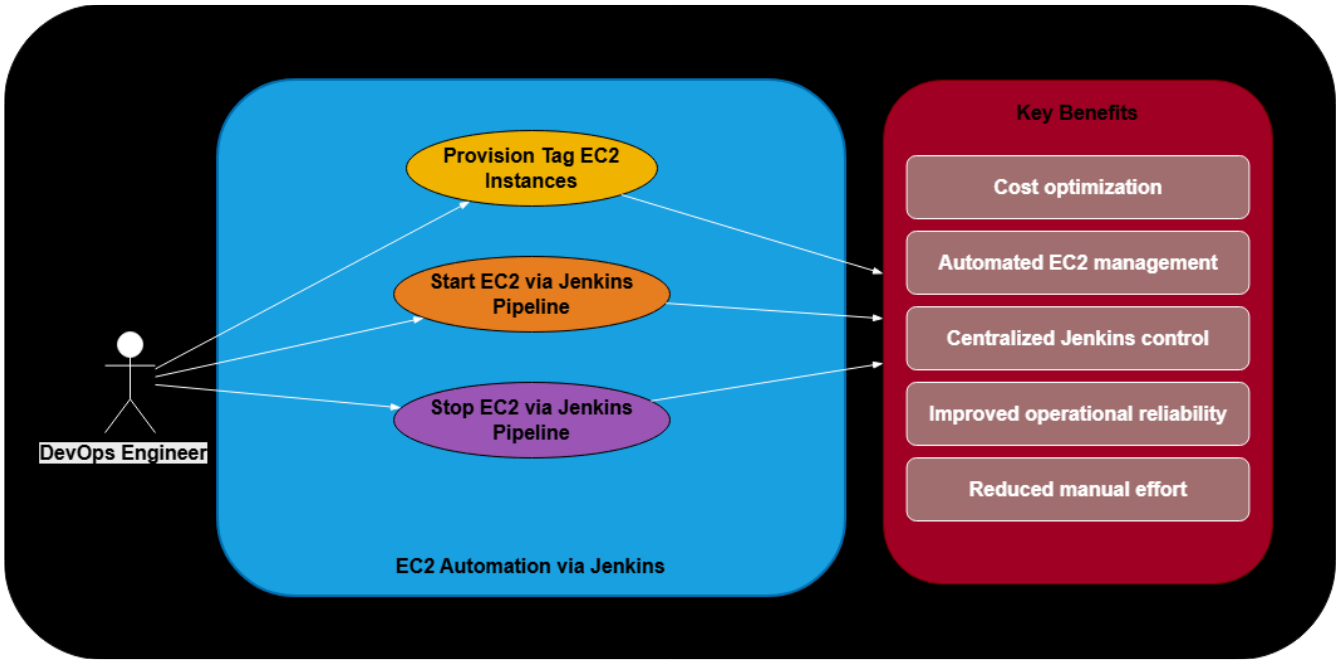
2.4 Constraints

- **Technology:** Must use Jenkins and AWS CLI.
- **Security:** Must adhere to IAM least-privilege principles.

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Start/Stop EC2 by Tags

- **Description:** Trigger a Jenkins job that identifies instances with specific `env` tags and changes their state.
- **Actors:** DevOps Engineer / Jenkins System.
- **Preconditions:** Instances exist with tags `env=dev` or `env=prod`. Jenkins has IAM permissions for `ec2:StartInstances` and `ec2:StopInstances`.
- **Postconditions:** Instance state matches the requested job (Running/Stopped).
- **Flow of Events:**
  a. User triggers "Start-Dev-EC2" job.
  b. Jenkins queries AWS for IDs where `tag:env=dev`.
  c. Jenkins issues `start-instances` command.
  d. Jenkins waits for `instance-running` status.
- **Alternative Flows:** If no instances are found with the tag, the pipeline logs an error and terminates.

3.2.2 Use Case 2: Scale ECS Fargate Service (Bonus)

- **Description:** Adjust Fargate `desired-count` via Jenkins pipeline.
- **Actors:** DevOps Engineer.
- **Preconditions:** ECS Cluster and Service are active.
- **Postconditions:** Desired count is updated to 0 (Stop) or 1+ (Start).
- **Flow of Events:**
  a. Jenkins executes `aws ecs update-service`.
  b. Parameters include `--cluster` and `--desired-count`.

---

4. Non-Functional Requirements

- **4.1 Performance:**

  Pipeline execution should complete within 3 minutes (including AWS "wait" times).
- **4.2 Security:**

  No hardcoded Secret Keys; use IAM Roles or Jenkins Credentials Manager.
- **4.3 Usability:**

  Pipelines should be parameterized so a single job can handle different environments (Dev vs. Prod).

- **4.4 Reliability:**

  Pipelines must include error handling to notify if an AWS API call fails.

---

5. Assumptions and Dependencies

- **5.1 Assumptions:** The Jenkins server has network connectivity to AWS public APIs.
- **5.2 Dependencies:** Success depends on the AWS CLI being correctly installed and configured on the Jenkins agent.

---

6. Risk Analysis

- **6.1 Risk:** Accidental shutdown of Production instances.
- **6.2 Mitigation:** Implement strict IAM policies and separate Jenkins jobs/folders with restricted access for `env=prod`.

---

7. Glossary

- **Tag-Based:** Using metadata (Key/Value pairs) to filter and target resources.
- **Desired Count:** The number of simultaneous instances of a task to run in an ECS service.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____